python笔记

2026年3月24日 16:00

is运算符检查两个变量是否指向同一个对象（内存地址比较）

Python中的赋值、浅拷贝和深拷贝：
直接赋值= ：变量b和a指向同一个对象，修改a会影响b。
浅拷贝（如dict.copy()或者切片）：创建新对象，但只复制第一层引用。如果元素是可变对象（如字典、列表），修改原对象会影响拷贝对象。
深拷贝（如copy.deepcopy(dict)）：完全复制对象及其所有嵌套对象，修改原对象不会影响拷贝对象。

.deepcopy()是深拷贝，即将被拷贝对象的全部内容递归拷贝到***新的内存空间***中，即自立门户互不影响，所以被拷贝对象怎么改变值也不会影响拷贝对象。

.copy()指的就是**浅拷贝；= 赋值也相当于浅拷贝**。

Map() # map函数，将指定的函数应用与一个或多个课迭代对象的所欲元素。
```
def double(x): return x * 2
numbers = [1, 2, 3, 4, 5]
doubled = map(double, numbers)
print(list(doubled)) # 输出: [2, 4, 6, 8, 10]
```

str.strip() # 用于移除字符串首尾的指定字符，默认为空白字符：\t\n 等。

Str.lstrip() # 只移除左侧字符。

Str.rstrip() # 只移除右侧字符。

Strs. capitalize() # 仅首个单词大写，例如：text = "hello world"

print(text.capitalize()） # 输出: "Hello world" (只有第一个单词大写)

str.lower() **# 将strs字符串的内容全部小写.**

str.casefold()方法（更激进的小写转换），适用于国际化文本。

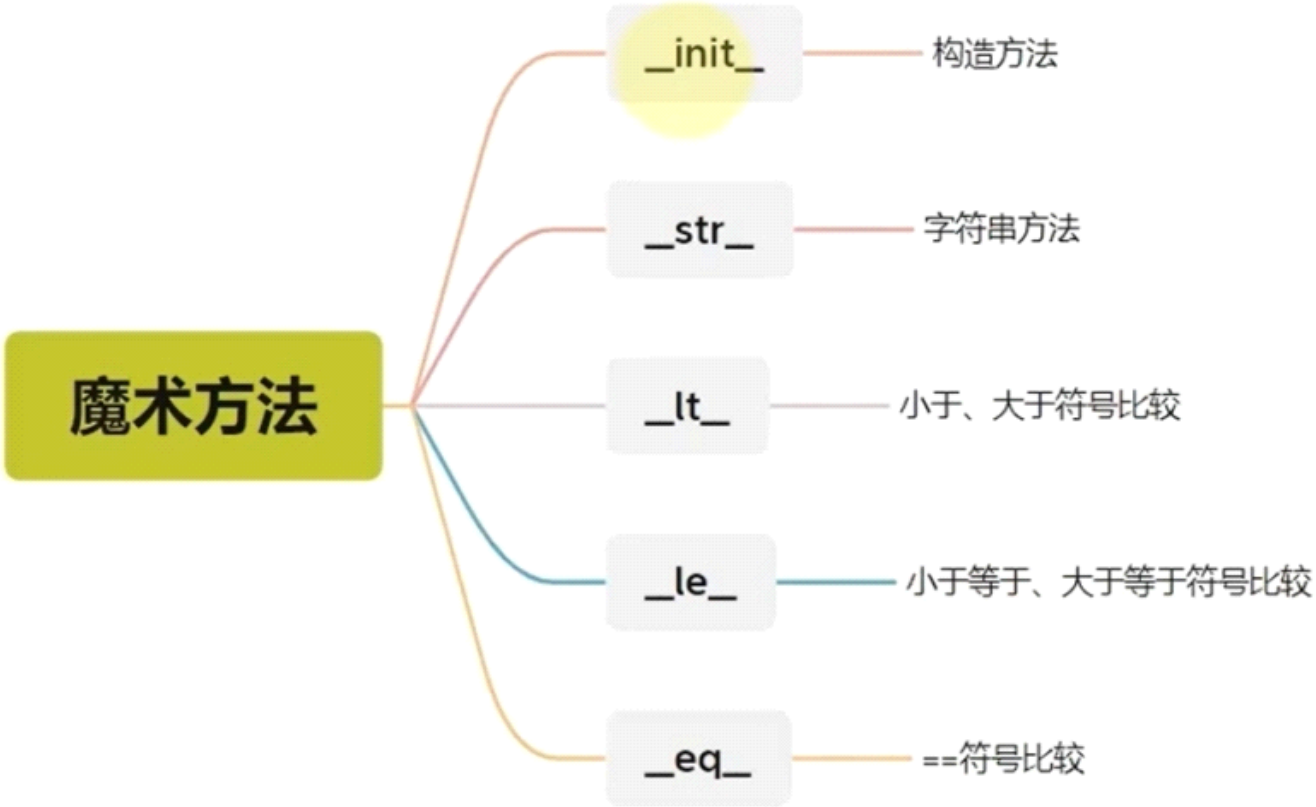
strs.upper() # 将strs字符串的内容全部大写.

strs.title()将字符串中每个单词的**首字母大写**，**其余字母小写**.以空格、标点等非字母字符分隔.
strs.upper().title() # strs.upper()- 首先执行,然后对upper()执行.title()操作。

异步编程是一种编程范式，**允许程序在等待操作完成时执行其他任务**，从而提高效率，特别是在I/O密集型和高并发场景中。

divmod(a, b)
a：被除数（数字类型，如整数、浮点数），**b：除数**（非零数字）。
结果：返回一个元组（**商，余数**），其中商 = a // b（向下取整的除法结果），余数 = a % b。

计算均值：np.mean(axis=0)
计算协方差矩阵：np.cov( )
中位数计算：np.median()
Spearman相关矩阵：df.corr(method=’spearman’) #pandas内置方法
pearson相关矩阵：df.corr(method=’pearson’) #pandas内置方法



方法	功能
__init__	构造方法，可用于创建类对象的时候设置初始化行为
__str__	用于实现类对象转字符串的行为
__lt__	用于2个类对象进行小于或大于比较
__le__	用于2个类对象进行小于等于或大于等于比较
__eq__	用于2个类对象进行相等比较

3. 空白字符包括哪些

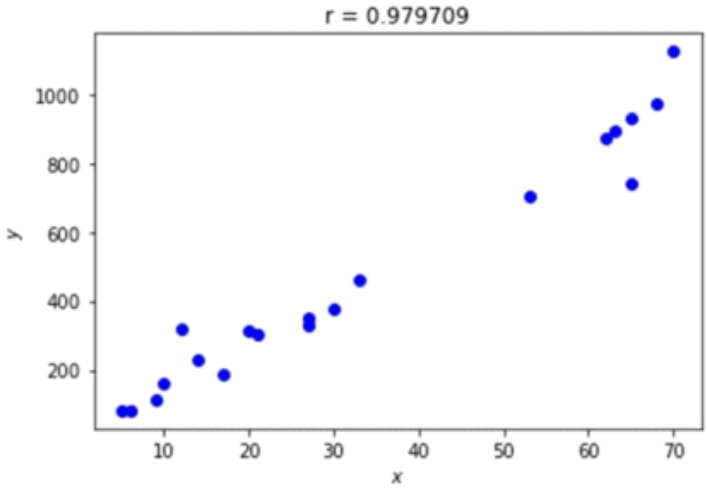
Python 中的空白字符包括:

- 空格 `' '`
- 制表符 `'\t'`
- 换行符 `'\n'`
- 回车符 `'\r'`
- 换页符 `'\f'`
- 垂直制表符 `'\v'`

例 1.9 对于 20 个随机选取的黄麻个体植株，记录其青植株重量 Y 与它们的干植株重量 X. 设二维总体 $\mathbf{X} = (X, Y)^{\mathrm{T}}$ 服从二维正态分布，其观测数据如下表:

(2) 计算 Pearson 相关系数并检验假设

$$H_0 : \rho_{XY} = 0 \quad \leftrightarrow \quad H_1 : \rho_{XY} \neq 0$$



36

(2) 计算 Pearson 相关系数并检验假设

$$H_0 : \rho_{XY} = 0 \quad \leftrightarrow \quad H_1 : \rho_{XY} \neq 0$$

算得数据的 Pearson 相关系数为 $r_{xy} = 0.97971$，

统计量 $t = \dfrac{r_{xy}\sqrt{n-2}}{\sqrt{1-r_{xy}^2}} \sim t(n-2)$　　统计量 $t_0 = 20.7387$

检验 p 值 $p = 2.58 \times 10^{-14}$，故 X 与 Y 的相关性是高度显著的.

```
# x 和 y 之间的 Pearson 相关系数
r = stats.pearsonr(x, y)
r
```

```
(0.9797090833693793, 5.145181377801077e-14)
```

练习：
自己写代码计算

37

(3) 计算 Spearman 相关系数并检验上述假设.

算得数据的 Spearman 相关系数为 $q_{xy} = 0.97366$，

统计量 $t = \dfrac{q_{xy}\sqrt{n-2}}{\sqrt{1-q_{xy}^2}} \sim t(n-2)$　　统计量 $t_0 = 18.1192$

检验 p 值为 $p = 2.63 \times 10^{-13}$，同样得 X 与 Y 的相关性是高度显著的.

```
# x 和 y 之间的 Spearman 相关系数
r = stats.spearmanr(x, y)
r
```

```
SpearmanrResult(correlation=0.9736646849622512, pvalue=5.260042458078761e-13)
```

(3) 分析各指标间的相关性.

```
# Pearson相关矩阵及显著性检验
pearson_p = np.zeros((data.shape[1], data.shape[1]))
for i in range(data.shape[1]):
    for j in range(i+1, data.shape[1]):
        rij = stats.pearsonr(data.iloc[:, i], data.iloc[:, j])
        pearson_p[i,j] = rij[1]
print(pearson_p)
```

```
[[0.000e+00 2.333e-06 1.128e-01 8.942e-02 2.717e-02 3.374e-01]
 [0.000e+00 0.000e+00 1.899e-01 1.650e-02 4.822e-03 4.850e-01]
 [0.000e+00 0.000e+00 0.000e+00 5.261e-01 3.401e-01 8.838e-01]
 [0.000e+00 0.000e+00 0.000e+00 0.000e+00 6.585e-04 2.622e-02]
 [0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 1.252e-03]
 [0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00]]
```

取显著水平 $\alpha = 0.1$，则 $r_{13}, r_{16}, r_{23}, r_{26}, r_{34}, r_{35}, r_{36}$ 的 p 值都大于α，

故认为各相应随机变量对的相关性很小，

相关系数可认为是0.

```
#spearman相关矩阵及显著性检验
spearman_p=np.zeros((df.shape[1],df.shape[1]))
for i in range(df.shape[1]):
    for j in range(i+1,df.shape[1]):
        rij=stats.spearmanr(df.iloc[:,i],df.iloc[:,j])
        spearman_p[i,j]=rij[1]
print(spearman_p)
```

```
[[0.00000000e+00 2.07035536e-05 4.93361598e-02 5.13801108e-02]
 [0.00000000e+00 0.00000000e+00 1.78878482e-02 2.46757038e-02]
 [0.00000000e+00 0.00000000e+00 0.00000000e+00 5.21001443e-04]
 [0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]]
```

```python
#pearson相关矩阵及显著性检验
pearson_p=np.zeros((df.shape[1],df.shape[1]))
for i in range(df.shape[1]):
    for j in range(i+1,df.shape[1]):
        rij=stats.pearsonr(df.iloc[:,i],df.iloc[:,j])
        pearson_p[i,j]=rij[1]
print(pearson_p)
```

```
[[0.00000000e+00 5.15283833e-05 8.48380741e-02 1.35839682e-01]
 [0.00000000e+00 0.00000000e+00 5.35790397e-02 1.31150557e-01]
 [0.00000000e+00 0.00000000e+00 0.00000000e+00 3.14055792e-03]
 [0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]]
```

运算符	描述	实例
+	加	两个对象相加 a + b 输出结果 30
-	减	得到负数或是一个数减去另一个数 a - b 输出结果 -10
*	乘	两个数相乘或是返回一个被重复若干次的字符串 a * b 输出结果 200
/	除	b / a 输出结果 2
//	取整除	返回商的整数部分 9//2 输出结果 4 , 9.0//2.0 输出结果 4.0
%	取余	返回除法的余数 b % a 输出结果 0
**	指数	a**b 为10的20次方， 输出结果 100000000000000000000

常用快捷键

- ctrl + alt + s：打开软件设置
- ctrl + d： 复制当前行代码
- shift + alt + 上\下： 将当前行代码上移或下移
- crtl + shift + f10：运行当前代码文件
- shift + f6： 重命名文件
- ctrl + a：全选
- ctrl + c\v\x：复制、粘贴、剪切
- ctrl + f：搜索

运算符	描述	实例
=	赋值运算符	把 = 号右边的结果 赋给 左边的变量，如 num = 1 + 2 * 3，结果num的值为7

复合赋值运算符

运算符	描述	实例
+=	加法赋值运算符	c += a 等效于 c = c + a
-=	减法赋值运算符	c -= a 等效于 c = c - a
*=	乘法赋值运算符	c *= a 等效于 c = c * a
/=	除法赋值运算符	c /= a 等效于 c = c / a
%=	取模赋值运算符	c %= a 等效于 c = c % a
**=	幂赋值运算符	c **= a 等效于 c = c ** a
//=	取整除赋值运算符	c //= a 等效于 c = c // a

它是将等号右边的结果

黑马程序员

Python中，其实支持非常多的数据类型占位

最常用的是如下三类

格式符号	转化
%s	将内容转换成字符串，放入占位位置
%d	将内容转换成整数，放入占位位置
%f	将内容转换成浮点型，放入占位位置

编号	使用方式	作用
1	列表.append(元素)	向列表中追加一个元素
2	列表.extend(容器)	将数据容器的内容依次取出，追加到列表尾部
3	列表.insert(下标, 元素)	在指定下标处，插入指定的元素
4	del 列表[下标]	删除列表指定下标元素
5	列表.pop(下标)	删除列表指定下标元素
6	列表.remove(元素)	从前向后，删除此元素第一个匹配项
7	列表.clear()	清空列表
8	列表.count(元素)	统计此元素在列表中出现的次数
9	列表.index(元素)	查找指定元素在列表的下标 找不到报错ValueError
10	len(列表)	统计容器内有多少元素

编号	方法	作用
1	index()	查找某个数据，如果数据存在返回对应的下标，否则报错
2	count()	统计某个数据在当前元组出现的次数
3	len(元组)	统计元组内的元素个数

编号	操作	说明
1	字符串[下标]	根据下标索引取出特定位置字符
2	字符串.index(字符串)	查找给定字符的第一个匹配项的下标
3	字符串.replace(字符串1, 字符串2)	将字符串内的全部字符串1，替换为字符串2 不会修改原字符串，而是得到一个新的
4	字符串.split(字符串)	按照给定字符串，对字符串进行分隔 不会修改原字符串，而是得到一个新的列表
5	字符串.strip() 字符串.strip(字符串)	移除首尾的空格和换行符或指定字符串
6	字符串.count(字符串)	统计字符串内某字符串的出现次数
7	len(字符串)	统计字符串的字符个数

编号	操作	说明
1	字典[Key]	获取指定Key对应的Value值
2	字典[Key] = Value	添加或更新键值对
3	字典.pop(Key)	取出Key对应的Value并在字典内删除此Key的键值对
4	字典.clear()	清空字典
5	字典.keys()	获取字典的全部Key，可用于for循环遍历字典
6	len(字典)	计算字典内的元素数量

数据容器分类

数据容器可以从以下视角进行简单的分类：

- 是否支持下标索引
  - **支持：列表、元组、字符串 – 序列类型**
  - **不支持：集合、字典 – 非序列类型**
- 是否支持重复元素：
  - **支持：列表、元组、字符串 – 序列类型**
  - **不支持：集合、字典 – 非序列类型**
- 是否可以修改
  - **支持：列表、集合、字典**
  - **不支持：元组、字符串**

	列表	元组	字符串	集合	字典
元素数量	支持多个	支持多个	支持多个	支持多个	支持多个
元素类型	任意	任意	仅字符	任意	Key：Value Key：除字典外任意类型 Value：任意类型
下标索引	支持	支持	支持	不支持	不支持
重复元素	支持	支持	支持	不支持	不支持
可修改性	支持	不支持	不支持	支持	支持
数据有序	是	是	是	否	否
使用场景	可修改、可重复的一批数据记录场景	不可修改、可重复的一批数据记录场景	一串字符的记录场景	不可重复的数据记录场景	以Key检索Value的数据记录场景

数据容器的通用统计功能

除了遍历这个共性外，数据容器可以通用非常多的功能方法

len(容器)

统计容器的元素个数

```
my_list = [1, 2, 3]
my_tuple = (1, 2, 3, 4, 5)
my_str = "itiheima"

print(len(my_list))     # 结果3
print(len(my_tuple))    # 结果5
print(len(my_str))      # 结果7
```

max(容器)

统计容器的最大元素

```
my_list = [1, 2, 3]
my_tuple = (1, 2, 3, 4, 5)
my_str = "itiheima"

print(max(my_list))     # 结果3
print(max(my_tuple))    # 结果5
print(max(my_str))      # 结果t
```

min(容器)

统计容器的最小元素

```
my_list = [1, 2, 3]
my_tuple = (1, 2, 3, 4, 5)
my_str = "itiheima"

print(min(my_list))     # 结果1
print(min(my_tuple))    # 结果1
print(min(my_str))      # 结果a
```

功能	描述
通用for循环	遍历容器（字典是遍历key）
max	容器内最大元素
min()	容器内最小元素
len()	容器元素个数
list()	转换为列表
tuple()	转换为元组
str()	转换为字符串
set()	转换为集合
sorted(序列, [reverse=True])	排序，reverse=True表示降序 得到一个排好序的列表

1. 匿名函数使用lambda关键字进行定义

2. 定义语法：

lambda 传入参数：函数体(一行代码)

3. 注意事项：

- 匿名函数用于临时构建一个函数，只用一次的场景
- 匿名函数的定义中，函数体只能写一行代码，如果函数体要写多行代码，不可用lambda匿名函数，应使用def定义带名函数

mode常用的三种基础访问模式

模式	描述
r	以只读方式打开文件。文件的指针将会放在文件的开头。这是默认模式。
w	打开一个文件只用于写入。如果该文件已存在则打开文件，并从开头开始编辑，原有内容会被删除。 如果该文件不存在，创建新文件。
a	打开一个文件用于追加。如果该文件已存在，新的内容将会被写入到已有内容之后。 如果该文件不存在，创建新文件进行写入。

操作	功能
文件对象 = open(file, mode, encoding)	打开文件获得文件对象
文件对象.read(num)	读取指定长度字节 不指定num读取文件全部
文件对象.readline()	读取一行
文件对象.readlines()	读取全部行，得到列表
for line in 文件对象	for循环文件行，一次循环得到一行数据
文件对象.close()	关闭文件对象
with open() as f	通过with open语法打开文件，可以自动关闭

1. 操作文件需要通过open函数打开文件得到文件对象

2. 文件对象有如下读取方法：

- read()
- readline()
- readlines()
- for line in 文件对象

3. 文件读取完成后，要使用文件对象.close()方法关闭文件对象，否则文件会被一直占用

1. 为什么要捕获异常？

在可能发生异常的地方，进行捕获。当异常出现的时候，提供解决方式，而不是任由其导致程序无法运行。

2. 捕获异常的语法？

```
try:
    可能要发生异常的语句
except[异常 as 别名:]
    出现异常的准备手段
[else:]
    未出现异常时应做的事情
[finally:]
    不管出不出现异常都会做的事情
```

3. 如何捕获所有异常？

异常的种类多种多样，如果想要不管什么类型的异常都能捕获到，那么使用：

- except:
- except Exception:

获的相关内容讲解

1. 如何自定义模块并导入？

在Python代码文件中正常写代码即可，通过import、from关键字和导入Python内置模块一样导入即可使用。

2. __main__变量的功能是？

if __main__ == “__main__”表示，只有当程序是直接执行的才会进入if内部，如果是被导入的，则if无法进入

3. 注意事项

- 不同模块，同名的功能，如果都被导入，那么后导入的会覆盖先导入的
- __all__变量可以控制import *的时候哪些功能可以被导入

什么是第三方包

我们知道，包可以包含一堆的Python模块，而每个模块又内含许多的功能。
所以，我们可以认为：一个包，就是一堆同类型功能的集合体。

在Python程序的生态中，有许多非常多的第三方包（非Python官方），可以极大的帮助我们提高开发效率，如：

- 科学计算中常用的：numpy包
- 数据分析中常用的：pandas包
- 大数据计算中常用的：pyspark、apache-flink包
- 图形可视化常用的：matplotlib、pyecharts
- 人工智能常用的：tensorflow
- 等

这些第三方的包，极大的丰富了Pyth[illegible]效

首先呢什么是第三方包呢

但是由于是第三方，所以Python没有内置，所以我们需要安装它们才可以导入使用哦。

Python数据和Json数据的相互转化

- Python数据和Json数据的相互转化

```
# 导入json模块
import json

# 准备符合格式json格式要求的python数据
data = [{"name": "老王", "age": 16}, {"name": "张三", "age": 20}]

# 通过 json.dumps(data) 方法把python数据转化为了 json数据
data = json.dumps(data)

# 通过 json.loads(data) 方法把json数据转化为了 python数据
data = json.loads(data)
```